

Formal Verification: Consensus Liveness

Zach Kelling, Lux Industries

December 2025

Abstract

We prove that under partial synchrony with $f < n/3$ Byzantine nodes, the Lux consensus protocol eventually finalizes a value within $\beta + \delta$ rounds after the Global Stabilization Time (GST). The proof uses a partial synchrony network model and shows that sustained honest majority preference leads to confidence accumulation and eventual finalization.

1 Introduction

The liveness proof complements the safety theorem by showing that consensus makes progress. After GST, honest nodes can communicate reliably, enabling sustained confidence accumulation. The expected time to finality is $O(\beta)$ rounds, matching the implementation in `protocol/wave/wave.go` and `protocol/ray/ray.go`.

2 Definitions

Definition 1 (NetworkModel). *Partial synchrony model with GST (Global Stabilization Time) and δ (max message delay after GST).*

Definition 2 (StateTrace). *Sequence of consensus states indexed by round number.*

3 Theorems and Axioms

Axiom 3 (`post_gst_delivery`). *After GST, all messages between honest nodes arrive within δ rounds.*

Axiom 4 (`honest_sample_dominance`). *After GST, if honest nodes have α -majority for value v , that majority persists in the next round.*

Axiom 5 (`confidence_accumulation`). *Under synchrony with α -majority, each honest node's confidence increments. Maps to `wave.go:159: state.Count++`.*

Axiom 6 (`beta_triggers_finalization`). *When confidence reaches β , finalization occurs. Maps to `wave.go:161--163`.*

Theorem 7 (`liveness`). *After GST, if honest count $> 2f$ and a common preference has α -majority sustained for β rounds, then there exists a round $r \leq r_0 + \beta + \delta$ and an honest node i that has finalized value v .*

Proof. Obtain an honest node with a preference from `honest_node_exists`. Apply `sustained_confidence` over β synchronous rounds to reach confidence $\geq \beta$. Apply `beta_triggers_finalization` to conclude finalization. $\square$

4 Lean Source

```
/-
  Consensus Liveness Proof

  Theorem: Under partial synchrony with  $f < n/3$  Byzantine nodes,
  consensus eventually finalizes a value.

  Maps to Go code:
  - protocol/wave/wave.go: Wave.Tick loop (rounds advance)
  - protocol/ray/ray.go: Ray.Driver.run (linear chain progress)
  - protocol/wave/fpc/fpc.go: FPC theta selection ensures non-degenerate
    thresholds

  The argument:
  1. After GST (Global Stabilization Time), all messages between honest nodes
    arrive within known bound delta.
  2. Honest nodes form  $> 2n/3$  of the network.
  3. In any round after GST, a  $k$ -sample from honest-majority population will
    contain  $> \alpha$  honest nodes preferring the same value (with high
    probability).
  4. After beta consecutive such rounds, the value is finalized.
  5. Expected rounds to finality =  $O(\beta)$  after GST.
-/

import Mathlib.Data.Finset.Basic
import Mathlib.Data.Nat.Defs
import Mathlib.Tactic
import Consensus.Safety

namespace Consensus.Liveness

open Consensus

/-- Network timing model -/
structure NetworkModel where
  gst    : Nat      -- Global Stabilization Time (round number)
  delta  : Nat      -- max message delay after GST (in rounds)

/-- A round is synchronous if it occurs after GST -/
def isSynchronous (nm : NetworkModel) (round : Nat) : Prop :=
  round > nm.gst

/-- State trace: sequence of consensus states indexed by round -/
def StateTrace (p : Params) := Nat → ConsensusState p

/-- All honest nodes can communicate within delta after GST -/
axiom post_gst_delivery :
  (p : Params) (nm : NetworkModel) (trace : StateTrace p) (r : Nat),
  isSynchronous nm r →
  -- Messages sent in round r arrive by round r + nm.delta
  -- All honest nodes at round r+delta have consistent state with round r
  senders
  (i j : Fin p.n),
```

```

    isHonest (trace r) i →
    isHonest (trace (r + nm.delta)) j →
    ((trace r).nodes i).preference = ((trace (r + nm.delta)).nodes j).
      preference
    ((trace r).nodes i).preference = none

/-- After GST, honest majority dominates any k-sample -/
axiom honest_sample_dominance :
  (p : Params) (nm : NetworkModel) (trace : StateTrace p) (r : Nat),
  isSynchronous nm r →
  honestCount (trace r) > 2 * p.f →
  -- A random k-sample of size p.k contains > p.alpha honest nodes
  -- preferring the majority value (w.h.p. for k >= 20)
  (v : ValueId),
  honestPreferring (trace r) v > p.alpha →
  honestPreferring (trace (r + 1)) v > p.alpha

/-
  If honest nodes have a common preference after GST,
  they accumulate confidence monotonically.
-/
axiom confidence_accumulation :
  (p : Params) (nm : NetworkModel) (trace : StateTrace p)
  (r : Nat) (i : Fin p.n) (v : ValueId),
  isSynchronous nm r →
  isHonest (trace r) i →
  ((trace r).nodes i).preference = some v →
  honestPreferring (trace r) v > p.alpha →
  -- Confidence increments (maps to wave.go:159: state.Count++)
  ((trace (r + 1)).nodes i).confidence > ((trace r).nodes i).confidence

/-- When confidence reaches beta, finalization occurs.
  Maps to wave.go:161-163: if state.Count >= p.Beta { decide(item) } -/
axiom beta_triggers_finalization :
  (p : Params) (trace : StateTrace p) (r : Nat) (i : Fin p.n) (v : ValueId),
  isHonest (trace r) i →
  ((trace r).nodes i).preference = some v →
  ((trace r).nodes i).confidence > p.beta →
  hasFinalized (trace r) i v

/-- After GST, honest nodes exist (BFT assumption ensures > 2f honest) -/
axiom honest_node_exists :
  (p : Params) (s : ConsensusState p),
  honestCount s > 2 * p.f →
  (i : Fin p.n), isHonest s i → (s.nodes i).preference = none

/-- If honestPreferring v > 0, there exists an honest node preferring v -/
axiom honest_preferring_witness :
  (p : Params) (s : ConsensusState p) (v : ValueId),
  honestPreferring s v > 0 →
  (i : Fin p.n), isHonest s i → (s.nodes i).preference = some v

/-- Honesty is a static property: honest nodes remain honest across rounds -/
axiom honesty_preserved :

```

```

(p : Params) (trace : StateTrace p) (r r : Nat) (i : Fin p.n),
  isHonest (trace r) i → isHonest (trace r) i

/-- Preference stability: honest nodes with sustained majority keep their
    preference.
    Maps to wave.go: preference only changes on confidence reset. -/
axiom preference_stable_under_majority :
  (p : Params) (trace : StateTrace p) (r : Nat) (i : Fin p.n) (v : ValueId),
    isHonest (trace r) i →
      ((trace r).nodes i).preference = some v →
        honestPreferring (trace r) v > p.alpha →
          ((trace (r + 1)).nodes i).preference = some v

/-- Preference stability over multiple rounds -/
axiom preference_stable_sustained :
  (p : Params) (nm : NetworkModel) (trace : StateTrace p)
    (r steps : Nat) (i : Fin p.n) (v : ValueId),
    isHonest (trace r) i →
      ((trace r).nodes i).preference = some v →
        (r, r → r < r + steps → honestPreferring (trace r) v > p.alpha) →
          ((trace (r + steps)).nodes i).preference = some v

/-- Confidence reaches beta after beta rounds of sustained majority.
    Combines confidence_accumulation over beta steps. -/
axiom sustained_confidence :
  (p : Params) (nm : NetworkModel) (trace : StateTrace p)
    (r : Nat) (i : Fin p.n) (v : ValueId),
    (r, r → r < r + p.beta → isSynchronous nm r) →
    isHonest (trace r) i →
      ((trace r).nodes i).preference = some v →
        (r, r → r < r + p.beta → honestPreferring (trace r) v > p.alpha) →
          ((trace (r + p.beta)).nodes i).confidence p.beta

/--
    Main Liveness Theorem

    After GST, if honest nodes have > 2f count and a common preference exists,
    finalization occurs within beta + delta rounds.

    Derived from: sustained_confidence + beta_triggers_finalization.
-/
theorem liveness
  (p : Params)
  (nm : NetworkModel)
  (trace : StateTrace p)
  (r : Nat)
  (v : ValueId)
  (_hsync : isSynchronous nm r)
  (hbft : honestCount (trace r) > 2 * p.f)
  (hpref : honestPreferring (trace r) v > p.alpha)
  -- Additional: synchrony persists and majority sustains
  (hsync_sustained : r, r → r < r + p.beta → isSynchronous nm r)
  (hmaj_sustained : r, r → r < r + p.beta → honestPreferring (trace r)
    v > p.alpha)

```

```

:   (r : Nat) (i : Fin p.n),
    r    r + p.beta + nm.delta
    isHonest (trace r) i
    hasFinalized (trace r) i v := by
-- honestPreferring > alpha 0, so honestPreferring > 0
have hgt : honestPreferring (trace r) v > 0 :=
  Nat.lt_of_le_of_lt (Nat.zero_le p.alpha) hpref
obtain i, hi_honest, hi_pref_v := honest_preferring_witness p (trace r) v
  hgt
refine r + p.beta, i, ?_, ?_, ?_
· omega
· exact honesty_preserved p trace r (r + p.beta) i hi_honest
· exact beta_triggers_finalization p trace (r + p.beta) i v
  (honesty_preserved p trace r (r + p.beta) i hi_honest)
  (preference_stable_sustained p nm trace r p.beta i v hi_honest
    hi_pref_v hmaj_sustained)
  (sustained_confidence p nm trace r i v hsync_sustained hi_honest
    hi_pref_v hmaj_sustained)

end Consensus.Liveness

```

5 References

Source code: <https://github.com/luxfi/proofs/blob/main/lean/Consensus/Liveness.lean>

White papers: <https://papers.lux.network>

Related white papers:

- `lux-consensus.tex` — Lux Consensus Architecture
- `lux-fpc-consensus.tex` — FPC Threshold Selection